



上下文与记忆管理

从 "记得住这一次" 到 "记得住这个人"

记忆管理

DK

为什么需要上下文与记忆管理：所有东西塞进 prompt 行不通

第七章之后还剩三个问题

1

上一版整包重发

能续修，但上下文越来越重
缺：选择和压缩策略

2

第五次还从零开始

系统不认识回头客
缺：跨会话偏好记录

3

新记忆继续全塞

长度上涨、重点被淹没
缺：预算与淘汰顺序

第七章让系统听懂了 " 这一句 "，这一章让系统记得住 " 这个人 "

它不是完全没有上下文，而是上下文还没有被管理

有了上下文与记忆管理之后

维度	没有	有了之后
多轮续修	上一版全文重发	结构化摘要 + 返工意见
个性化	每次重新声明偏好	系统已知偏好，默认匹配
成本	多说一遍 = 多烧一轮	省掉澄清轮次
信任	" 每次像跟新客服讲一遍 "	" 它记得我上次的选择 "

归根到底是一条：减少用户和系统之间重复沟通的次数

意图层和生成层是前后两站，不要混在一起

上下文 vs 记忆 · 先把概念分清楚

	上下文 Context	记忆 Memory
回答什么	"这次 prompt 里放什么"	"跨次 / 跨会话保存什么"
生命周期	一次请求（组装完即消费）	持久化（有淘汰策略）
核心约束	token 预算	存储 + 检索成本
主要操作	选择、压缩、排序	写入、晋升、遗忘
类比	今天这顿饭的菜单	冰箱里的食材

记忆是上下文的素材库，上下文是记忆的消费者

分清 " 存了没有 " 和 " 取了没有 "，是定位问题的第一步

实现落到项目哪三块核心

1

TravelPlanningEngine

外层 Loop 的心脏
plan()→runLoop() 的 for 轮次
这一章动的是 " 打包材料 " 这步

2

PlanChatService.buildPrompt()

每轮 prompt 在这拼
四段标记: INSTRUCTION /
CONSTRAINT / HISTORY /
PROFILE

3

PlanController + Session

POST
/api/plan/ask→202+sessionId
SSE 推事件; Session 存本次
events/versions, 跨会话画像还没有

HISTORY 现在还是上一版整包 JSON——先把欠账晒出来, 下一节再还

实现完，当场跑一遍看效果

1

启后端

cd backend → mvn compile spring-boot:run ; 模型 qwen3.8-flash , 端口 8080 ; 先设好 DASHSCOPE_API_KEY , 没设直接拒启

2

提交 " 杭州三天 "

产品页 localhost:5173 提交, 看 SSE 事件逐条推: 生成→契约校验→约束校验→反馈准备, 外层 Loop 在转

3

翻 PromptDumper 日志

打开 logs/prompts/{sessionId}-{round}.txt 末尾的分区报告, 现场看总长 / 历史区 / 画像区占比

当场印证两件事

① 历史区续修轮带的还是上一版整包 JSON—— 这笔账下一节还; ② 画像区现在是 0—— 跨会话记忆要到第三节才接进来。重点不是某个漂亮数字, 而是从此每次取舍有账可查。

记录的是交给 ChatClient 的文本, 不含 tool schema , 不等于供应商精确账单

本章路线图 · 两个缺口四节补

1

8.2 上下文工程

prompt 怎么组装——选什么、压什么、什么顺序

2

8.3 四层记忆 + 偏好学习

跨会话怎么记——存什么、怎么学、什么时候遗忘

3

8.4 测试、评测与验收

记忆系统怎么验——八维自动化评测 + 三维人工感受

4

8.5 本章小结

判据回顾 + 还缺什么 + 下一章预告

每一节还是同一个套路：先定判据 → 再做方案 → 再写 Prompt → 最后读代码

本节小结

第七章让系统听懂了 " 这一句 "，这一章让系统记得住 " 这个人 " 和 " 刚才那段对话 "。

上下文是菜单，记忆是冰箱——分清这两件事，是定位问题的第一步。

回到主线

三个问题（整包重发 / 从零开始 / 继续全塞）指向两个缺口：prompt 缺预算管理，跨会话缺用户画像。

你带走的能力

会区分上下文和记忆：一个管 " 这次带什么 "，一个管 " 跨次存什么 "。分清了才能在出错时定位到正确的层。

上下文工程—— Prompt 里该放什么、怎么放

为什么不能 " 全塞进去 "

1

单次也需要预算

长度跟天数 / 活动 / 反馈有关
组装器必须知道上限和淘汰顺序

2

信息太多反而抓不住

Lost in the Middle (Liu 2023)
越长，中段越危险

3

钱和时间跟长度走

多轮 Loop 反复发送
冗余按轮次重复支付

一条信息值不值得放进 prompt，看它会不会改变模型这一次的输出。不改变就不放

这是上下文工程的第一性原则

Prompt 四分区与优先级

分区	装什么	优先级	淘汰规则
指令区	系统角色 + 输出要求	最高	永不淘汰
约束区	budget/days/mustVisit	高	永不淘汰
历史区	上一版摘要 + 返工意见	中	超预算时压缩
画像区	用户长期偏好	低	超预算时优先丢弃

越接近 " 这一次的硬约束 " 优先级越高, 越是 " 历史推断 " 越低

先砍画像, 再压历史, 指令和约束永远不动

指令区 • 固定前缀永不压缩

压了会怎样

破坏输出契约

模型可能返回无法解析的内容

→ 下一轮仍要返工

固定的好处

稳定前缀可命中缓存

更容易对账

→ 不要塞时间戳等动态内容

约束区同样不能砍

用户说 "必去雷峰塔" 你把它丢了 → 功能性 bug，验证器拦住再跑一轮

你以为省了 token，其实多花了一整轮

淘汰顺序：先砍画像 → 再压历史 → 指令和约束永远不动

砍掉约束不是省 token，是多花一整轮

放在哪 • 位置效应与物理排布

决定什么

依据

分区优先级

放不放、超预算先砍谁

会不会改变这一次的输出

物理顺序

排在哪个位置

越关键越靠首尾

- [1] 指令区 ← 最前面，固定前缀
- [2] 约束区 ← 紧跟指令，硬需求要早
- [3] 历史区 ← 居中，最能牺牲的放最差位置
- [4] 画像区 ← 靠后，条数少每条都相关
- [5] 任务复述 ← 收尾，有意重复硬约束

关键约束占头尾两端，可牺牲的历史让给中段

四种压缩手段 · 从轻到重

1

选择性注入

本次已写死的字段不再被画像覆盖，条件不匹配的不进来

2

摘要替代原文

完整 JSON → 结构化摘要（航班 / 酒店 / 景点）

3

滑动窗口

只保留最近 N 轮，当前项目天然只带 " 上一版 "

4

结构化替代自然语言

散文有歧义 → 键值对：更精确、可寻址、短一半

压缩的终点不是 " 更短的话 "，是 " 更结构化的数据 "

结构化 = 更精确 + 可寻址 + 短一半

一次续修 · 模型真正需要什么

场景：用户确认 " 预算改到 2600、雷峰塔保留 " → 意图层转成新 PlanRequest，才进规划 Loop

分区	这一轮真正要装什么
指令区	系统角色 + 事实纪律 + 输出要求
约束区	新预算 / 天数 / 目的地 / 必去景点 + 上轮必改项
历史区	上一版选中的航班 · 酒店 · 景点摘要，不发完整 dailyPlans JSON
画像区	本次为空——第三节才接入真实用户画像

固定回放实测（仅代表课程样例）

首轮 $\approx$ 416 token；带上一版摘要 + 两条返工意见的续修轮 $\approx$ 645 token。价值不在数字本身，而在——现在可以量、可以比、也知道哪一段被丢了。

上下文工程的本质不是 " 放得多 "，是 " 每一个 token 都在干活 "

预算从哪来 • 量错往哪边错

max-tokens = 6000 • 单页的尺子

looptrip.context.max-tokens

管单次 prompt 能组装多长

是 ContextAssembler 的排版尺

max-tokens = 20000 • 总账护栏

第六章那一个，别搞混

管整次规划累计烧多少钱

一个管总账，一个管单页

怎么量、为什么不能塞满

`ceil(content.length()/1.5)` 估算——中文 • JSON • 英文混排没有稳定系数，一定不准。

统一预留 15% 安全余量：配 6000 时只按 5100 装内容。

这把尺子只量 system/user 文本，不含框架附带的 tool schema，不是供应商精确计费器。

粗估值必须配安全余量——宁可少放一段画像，也不要把单次预算刚好塞满

不要把粗估值当精确上限：固定 1.5 系数 + 额外 15% 缓冲

Prompt R · 实现上下文组装器 ContextAssembler

Prompt R ·
ContextAssembler

继续改这个旅行规划项目。把 `buildPrompt()` 改成有预算意识的 `ContextAssembler`。

组装顺序：固定指令 → 硬约束 → 上一版摘要 → 用户画像 → 末尾复述
系统指令和硬约束不能丢；上一版改成结构化摘要。
上轮返工意见跟硬约束放一起，预算紧也必须保留。
不要调模型做摘要。

先建最小 `UserProfile`，本节传空画像。
空间不够先不放画像，再降级为最小摘要。
把放了 / 丢了哪些区段和原因放进 `PromptContext`。

`ContextAssembler` 无状态、不读 `session`、不做 IO；
不要放进 `TravelPlanningEngine`。
保留第一节的 `### SECTION` 分区标记——`PromptDumper` 的分区统计靠它切段。

AI 已经认识项目了——只需点出它会默认做错的关键约束

Prompt R · 三个必须明确说的点

" 不要调模型做摘要 "

否则形成递归调用
复杂度和成本不可控

" 记录被砍了什么 "

看不见的决策 = 不存在的决策
可观察性延续

" 不要改 Engine "

组装在 PlanChatService
更新旧断言契约

这三条不说，AI 默认都会做错： 它会用 LLM 做摘要、静默丢弃、把逻辑塞进 Engine 。

ContextAssembler • 先保什么再丢什么

ContextAssembler.java -

淘汰级联

```
int usableBudget = (int) Math.floor(
    tokenBudget * (1 - safetyMargin));

if (!fits(constraint, history, profileText, tail, usableBudget)) {
    dropped.add(new DroppedSection("profile", " 剩余预算不足 "));
    profileText = "";
    // 首轮没有历史区，第二级级联无从发生，不误报
    if (!history.isEmpty() && !fits(constraint, history,
        profileText, tail, usableBudget)) {
        dropped.add(new DroppedSection("history-detail", " 超出剩余预算 "));
        history = minimalHistory;
    }
}
```

重点不是 if，而是顺序：全放 → 去画像 → 压历史。指令 / 约束 / 返工永不淘汰

顺序：组装 → 记事件 → 落盘 → 调用模型

真实演示 · 跑两遍看上下文淘汰级联

1

第一遍：正常预算（max-tokens=6000）

提交 " 杭州三天 " → 看 SSE 里 CONTEXT_ASSEMBLED 事件：includedSections / droppedSections / estimatedTokens / usableBudget。正常预算下 dropped 应为空，included 无 HISTORY（首轮没有上一版）。同时打开 logs/prompts/ 看 PromptDumper 的分区报告。

2

第二遍：改小预算逼出淘汰（max-tokens=400）

改 application.yml → 重启 → 提交同样请求。 $400 \times 0.85 = 340$ ，首轮 416 token 装不下，droppedSections 必现 "profile: 剩余预算不足"；"history-detail: 超出剩余预算" 只在带历史摘要的返工轮出现。对比两次 PromptDumper 日志：指令 / 约束一直在，画像先丢，历史到返工轮再压缩。

CONTEXT_ASSEMBLED 事件 · 第八章新增的第一个事件类型

includedSections + droppedSections + estimatedTokens + usableBudget → 发生裁剪时能回答 " 这一轮为什么没带画像 "

演示完记得把 max-tokens 改回 6000

本节小结

上下文工程解决的是 " 这一次放什么 "—— 在 token 预算内组装最有效的 prompt。
指令和约束是底线，历史可以压缩，画像可以丢弃，但每一次取舍都必须被记录。

回到主线

四分区优先级（指令 > 约束 > 历史 > 画像） + 四种压缩手段 + ContextAssembler 无状态组装。

你带走的能力

会设计 prompt 的预算和淘汰策略：一条信息值不值得放，看它改不改变这一次的输出。

四层记忆与偏好学习——信息住在哪、怎么学

分层判据 · 按“活多久”分，不按“重不重要”分

层	名称	生命周期	存储位置	旅行规划实例
1	工作记忆	当前轮次	PlanningLedger	本轮方案、problems
2	会话记忆	当前会话	PlanningSession	版本链、requestDiff
3	用户记忆	跨会话持久	profiles/{userId}.json	已确认偏好
4	系统知识	全局静态	配置 / 事实快照	群体默认值

冰箱按保质期分格，记忆按**生命周期**分层——四种寿命，四个隔层

会话记忆是缓冲区：修改先观察，确认后才升入用户记忆

每一层怎么管 · 为什么不能只分两层

工作记忆

每轮覆盖
不用管

会话记忆

会话结束回收
第 7 章已建好

用户记忆

需要遗忘策略
只写不删会污染

系统知识

手动维护
兜新用户

归短期？

会话结束就丢
永远学不到偏好

归长期？

临时修改直接进永久存储
“这次带孩子不爬山” 变成 “永远不爬山”

层与层之间的晋升必须有显式条件，不能自动扩散

会话记忆是学习原料，达到门槛经用户确认才进 UserProfile

冷启动 · 新用户的用户记忆是空的

问题

新用户第一次进来
用户记忆是空的
画像区一个字填不出来
前几次靠谁兜？

答案：系统知识层

群体先验：每天 ≤ 4 景点
换乘 ≥ 40 分钟、用餐 11-14 点
固化在验收约束配置 `looptrip.constraints`

群体先验住系统知识层，个人偏好住用户记忆层。混在一层永远分不清

为什么不塞进用户记忆

它不是这个人的偏好。一旦混进画像，他要删你不知道该不该让他删——删的是偏好还是产品默认值？

" 系统猜的 " 和 " 用户说的 " 可信度和可删除性完全不同

晋升判据 · 什么时候从会话提升到用户记忆

核心问题

用户说 "预算从 2600 改到 3000"—— 他永远偏好 3000，还是这次去三亚比较贵？

错误做法：把所有修改直接写进用户记忆 → 一次性场景被永久化

1

频次门槛

同一修改模式

≥ 3 个不同会话

同会话反复改只算一次

避免把一次纠结当长期偏好

2

方向一致性

4/5 次改成民宿 = 强信号

2 次上调 2 次下调 = 浮动

频次够但方向不一致

说明随场景变化

3

上下文一致

"带孩子不爬山" 可以晋升

但必须连同条件一起确认

不能写成无条件的 "不爬山"

三个月后独自去黄山就完了

无论哪种候选，用户确认前都不能写入画像

先看频次，再看方向，有条件就带着条件确认

晋升判断表 · 五个例子过一遍

例子	修改模式	出现会话	方向	晋升结果	理由与动作
1	连锁酒店→民宿	4 个会话	一致	确认后写入	强偏好信号 · 无条件
2	预算加四百	5 次里 2 次	不一致	不能晋升	无稳定方向 · 场景依赖
3	删除爬山类景点	5 次里 1 次	—	不能晋升	频次不够 · 可能临时场景
4	带孩子时删爬山	3 个会话	一致	带条件确认	条件成立时才召回使用
5	行程删除购物点	4 个会话	一致	确认后写入	强偏好 · 无条件

逐行对照三条判据：频次够不够 → 方向稳不稳 → 条件带不带

判断表终点只有一个：等用户点头，候选才写入画像

召回 · 偏好多了不能全塞，要挑

问题

半年用户一两百条偏好
全塞进画像区 → 超预算
画像区整体丢弃 → 白学

解法：按相关性挑

不能全放，要挑
跟这一次请求相关的放进来
其余的留在冰箱里不动

1

字段相关

这次涉及哪个领域
就只召那个领域的偏好
餐饮偏好别放进节奏请求

2

条件匹配

"带孩子时不爬山"
不带孩子 → 不该召回
条件不成立还放进去更糟

3

时间新鲜

上周确认 > 三个月前
越新被触发的排越前
三道过滤 + 排序，取前 8 条

记忆不是越多越好，是越准越好——决定质量的不是存了多少条，是能不能准确召回该用的那几条

不挑就等于白学：学了半年的偏好，一条都没进 prompt

偏好冲突 · 画像说一回事，请求说另一回事

场景

画像: " 预算通常 2500-3000"

本次请求: " 预算 2600"

两条信息重叠了，听谁的？

规则

本次显式约束永远压过历史画像

用户都开口说了精确数字

历史推断就该让路

不需要额外写冲突解决器

上一节定的分区顺序本身就是冲突规则：约束区排在画像区之上。

召回偏好时把已被本次请求明确指定的字段跳过，覆盖就自动发生了。

画像内部两条自己打架呢？

" 总选民宿 " 和 " 接受连锁酒店 " 同时存在 → 不该指望召回去挑，它在写入时就不该被允许发生。

这就是删除为什么必须是一等公民。

记错的代价与确认策略

意图识别记错了

"修改预算" 误判为 "取消"

后果立刻可见

用户能即时纠正

记忆记错了

"不爬山" 写进画像

永远不推荐登山

用户不知道为什么

策略	适用场景	实例	理由
隐式推断	记错代价低	展示顺序偏好	错了下次覆盖
显式确认	记错代价高	"以后默认民宿？"	后续全部受影响
带条件标签	有上下文依赖	"带孩子时不爬山"	条件消失后不生效

记忆系统最危险的属性不是 "记不住"，是 "忘不掉"

遗忘必须是一等公民操作

可解释、可查看、可删除 · 三件套缺一不可

1

可解释

"你为什么这么推荐?"

缺了: 只能怀疑整个系统

2

可查看

"你都记了我什么?"

缺了: 不知道有什么可删

3

可删除

"把这条忘掉"

缺了: 知道错了也改不了

落地靠一个字段

每条偏好记录它从哪几次会话学来的 → 出示证据, 让用户自己判断留还是删

缺了前两件, 删除权就是空的——用户根本不知道有什么可删

偏好是个人数据

可导出 · 可清空 ("忘记我") · 可关闭 (关掉偏好学习)

遗忘和删除必须打审计日志

三件做不到, 你的记忆系统不该上线

Prompt S · 偏好学习与记忆管理

承接关系

第一节 Prompt 实现了 PromptDumper（落盘 + 分区计量），第二节 Prompt R 实现了 ContextAssembler（预算组装）。这一轮把“写入”和“遗忘”补上——ContextAssembler 已经会读画像，但画像一直是空列表。

Prompt S · 偏好学习

继续在这个仓库里做。先读 PlanningSession、修改版本、SSE 确认流程、ContextAssembler 和 UserProfile。

每次 RevisionPolicy 接受修改后留 RevisionRecord。学习时按修改类型汇总，同会话反复改只算一次。

PromotionPolicy: <3 会话→WAIT 方向不一致→REJECT
稳定候选→用户确认 带条件→连条件一起保存

用户确认前不进 UserProfile。

已确认偏好保存：内容 / 置信度 / 条件 / 时间 / 来源 sessions
最多召回 8 条；90 天未用降置信度→自动删除

增加 PREFERENCE_LEARNED / CONFIRMED / FORGOTTEN 三事件
相反修改直接删除旧记录；审计日志

按现有工程写：告诉 AI 从哪拿修改记录，候选 / 确认 / 画像分别住在哪

核心代码讲解 • 讲师看这里

1

PreferenceLearner.java — 只做汇总，不做决定

打开 `com.looptrip` 包下的 `PreferenceLearner`。讲 `analyze()` 方法（L22-32）：同类修改放到一组，同会话只保留最后一次，产出候选列表。重点是——它不决定候选能不能进入画像，频次不足的也保留下来，为的是让下一步的 `PromotionPolicy` 有真实输入。

2

PromotionPolicy.java — 真正的门槛在这里

紧接着打开同包的 `PromotionPolicy`。讲 `evaluate()` 方法（L12-19）的四个分支：频次不够返回 `WAIT`（继续观察）、方向不一致返回 `REJECT`、有条件的返回 `CONFIRM_WITH_CONDITION`、无条件的返回 `CONFIRM_UNCONDITIONAL`。告诉学员这就是前面讲三条判据在代码里的落地。

3

和 Loop 的关系

用户每次返工都会留下信号，但 `Loop` 不急着把一次返工当永久规则。先观察、再判断、最后让用户确认——长期记忆才不会污染后面每一轮。`Engine` 仍然只管 `Loop`，记忆在模型调用边界才注入 `Prompt`。

四层记忆 · 在跑着的系统里看

L
1

工作记忆

事件流 →
REVIEW_COMPLETED
problems /
constraintResults
只活在当前轮
下一轮直接覆盖

L
2

会话记忆

事件流 →
REVISION_STARTED
requestDiff / 版本链
活到会话结束
第七章已搭好

L
3

用户记忆

profiles/{userId}.json
偏好 / 置信度 / 条件
跨会话持久
后面的演示往这里写

L
4

系统知识

应用配置 / 事实快照
"每天 ≤ 4 景点" 等默认值
新用户靠它兜底
全局静态、运营维护

理论表格对应到真实文件——学员后面看偏好演示就知道数据往 L3 的哪个 JSON 里写

先发一个请求让四层都有数据，再开始偏好生命周期演示

真实演示 • 一条偏好走完整个生命周期

1

会话 1-2：酒店→民宿，观察期

两次新会话各提一次 " 改成民宿 "。事件流显示 PREFERENCE_LEARNED(WAIT)，频次分别为 1、2。页面不弹确认卡片——门槛未到，系统在安静观察。

2

会话 3：频次到 3，弹出确认

事件流决策变为 CONFIRM_UNCONDITIONAL。页面弹出 " 记住：住宿优先选择民宿 " 确认卡片。点击 " 记住 " 后 事件流显示 PREFERENCE_CONFIRMED。打开 data/profiles/course-demo-user.json 看写入结果。

3

会话 4：画像生效 + 相反修改删除

不做修改直接看 PromptDumper 快照——SECTION:PROFILE 从占位行变成真实偏好文本（PROFILE 一直在 includedSections 里，别盯它，token 也不一定变大）。再改成 " 住宿优先选择连锁酒店 "——事件流显示 PREFERENCE_FORGOTTEN，profiles JSON 里 " 民宿 " 消失。

返工不再只服务当前一轮，它可以变成下一次 Loop 的软性输入

观察、确认、衰减和删除共同保证这条输入不会失控

本节小结

冰箱管好了——四层各自的生死、怎么补货、怎么准确召回、过期的怎么扔，都有交代了。

记忆系统最容易出的问题不是 "装不上"，是装上了以为在生效，其实没生效或在反效。

回到主线

四层记忆（工作 / 会话 / 用户 / 系统） + 三条晋升判据
+ 相关性召回 + 可解释 · 可查看 · 可删除三件套

你带走的能力

会设计偏好的晋升门槛和遗忘策略：记错了代价多大决定用显式确认还是隐式推断。

记忆评测与验收——记对还不够，还得忘对

记忆故障 · 时间型的新挑战

故障	表现	为什么难发现
记错了	一次不爬山→永久标记	用户不知道系统替他决定
忘太早	3 次选民宿第 4 次从零	用户觉得 "笨" 分不清
忘不掉	说 " 试酒店 " 仍注入民宿	显式被隐式覆盖

它们不会在一次会话内暴露，**你必须跨越多个会话才能抓到**

验证手段也要对应调整——一部分自动跑，一部分靠人感受

两拨验证 • 各管什么

AI 自动化评测

问什么：记对了吗？忘对了吗？没串用户吗？

怎么跑：构造评测集，跑 prompt，检查输出

频率：改了记忆代码就跑

人工评测

问什么：被记住舒服还是诡异？覆盖时像配合还是争辩？

怎么跑：讲师在页面走一遍

频率：重大改动后手动走一遍

能算的不交给给人，人只负责机器判断不了的感受

AI 自动化评测 · 八个维度

维度	具体体验什么	达标线
预算丢弃顺序	画像先于历史先于约束被砍	100%
丢弃可追溯	droppedSections 包含原因	100%

维度	具体体验什么	达标线
偏好晋升正确性	3 次晋升 /2 次不晋升	100%
遗忘衰减正确性	3 次矛盾后旧偏好移除	100%
条件偏好尊重	带孩子才生效	100%
显式覆盖尊重	说 " 试酒店 " → 不注入民宿	100%
跨用户隔离	A 偏好不出现在 B 的 prompt 里	100%
个性化命中率	方案是否体现注入偏好	≥80%

记忆那六维度每条必须成对断言—— " 记住了 " 和 " 没乱记 " 同时验

" 该忘的没忘 " 比 " 该记的没记 " 更危险

评测集 · 50 画像 × 3 请求 = 150 条

50

用户画像

偏好民宿
预算敏感
带孩子不爬山
偏好自由行…

×3

每画像三请求

命中场景
偏好无关场景
显式覆盖场景

5

合计用例

跨会话序列
≈600 次模拟
跑一次 40 分钟

请求类型	具体内容	预期结果
命中	杭州 3 天 /2600/ 带孩子	方案有民宿、无登山
无关	上海 2 天 /5000/ 商务	偏好都不触发
覆盖	成都 4 天 /" 这次住酒店 "	显式压倒画像

实操陷阱

用例之间必须隔离 userId ， 否则前面偏好污染后面测试
不进 CI—— 测试每次提交跑， 评测按改动面触发

三类少了任何一类， 评测就有盲区

Prompt T · 实现自动化评测

Prompt T · 自动化评测

这一章做了上下文组装和记忆管理，建一套自动化评测。

1. 预算丢弃顺序：紧预算 → 画像先砍，约束保留
2. 丢弃可追溯：droppedSections 包含区段名和原因
3. 偏好晋升：3次→有 / 2次→无
4. 遗忘衰减：相反修改→旧偏好立即移除；时间衰减
5. 条件偏好：带孩子→不爬山 / 不带孩子→可爬山
6. 显式覆盖："试酒店"→本次不注入 / 下次恢复
7. 跨用户隔离：A 偏好不出现在 B 的 prompt
8. 个性化命中率：AI judge 跑 3 次取多数票

输出：每维度通过率 + 失败用例 → markdown 报告

前 7 维度

硬门槛

全部 100% 才放行

第 8 维度

软指标

看趋势不做卡点

AI judge 的分数只用来看趋势，卡点必须建立在确定性指标上

真实演示 · 跑评测看报告

1

前七维度：全部 100% 才正常

打开评测报告，前七个维度通过率应该全是 100%。如果不是——当场看失败用例排查原因。比如 "偏好晋升" 挂了一条，对着三条判据查，根因通常在频次计算或方向判断的边界条件上。

2

第八维度：命中率 + judge 投票明细

个性化命中率通常 80%~90%。打开一条 "未命中" 用例，看模型方案 vs 注入偏好——有时模型没遵循，有时偏好描述不够具体。这就是为什么它是软指标：模型有合理波动。

3

Judge 分歧用例：为什么要多数票

找一条 2:1 投票的用例，打开三次判断理由。让学员看到 AI 评委自己也有噪声——同一份方案，措辞稍有不同就影响判断。这是所有 AI 评测的标准操作。

以实际执行结果为准——让学员亲眼看到评测在跑、报告能读、失败能查

数字因模型版本和评测集内容而波动，趋势比绝对值重要

人工评测 · 在规划页面上验感受

维度	问什么	为什么机器判不了
被记住的感受	"记住我了"——贴心还是被监视	同功能不同人感受相反
覆盖时态度	说"试酒店"配合还是争辩	配合 / 争辩断言写不出来
解释可信度	"因为三次都选了民宿"——信不信	证据对但表达影响信任
演示用例故事线		
步骤 1-7：偏好学习（3次改民宿→第4次是否主动推荐）		
步骤 8-9：显式覆盖（"试酒店"→立刻配合）→恢复注入		

显式表达永远优先于隐式记忆，系统被覆盖时不许有态度

人工评测有明确用例、观察点、判断标准——结论是人的感受不是数字

三段小结 · 讲师怎么讲这个过程

关于 " 被记住 "

三次学会→还挺聪明
一次就记住→觉得被监视
晋升阈值宁可迟钝一点

关于 " 覆盖态度 "

正确反应：立刻配合不评论
错误反应：" 建议继续选民宿 "
→ 在跟用户争辩

关于 " 恢复信任 "

加一句 " 根据偏好推荐 "
→ 用户理解来源
可解释性是信任的基础

可解释性不是装饰品，它是信任的基础

和 AI 评测的区别只有一个：结论是人的感受，不是数字

本节小结

两拨验证：AI 自动化评测管八个维度（前七个硬门槛，第八个看趋势），人工评测管三个感受维度。

150 条用例 × 跨会话模拟，跑一次 40 分钟——只在改了记忆逻辑时才跑。

回到主线

八维度评测 + 人工三维度 + Prompt T 实现评测自动化。

"该忘的没忘" 比 "该记的没记" 更危险。

你带走的能力

会设计记忆系统的评测维度：确定性指标做卡点，AI judge 只看趋势。

成对断言是基本功。

本章小结——从 " 能对话 " 到 " 能进化 "

能力层次 · 第五层解锁

层次	达成于	你敢做什么	核心交付物
能跑	第 3-4 章	敢自己盯着跑	Loop+ 生成器 + 验证器 + 四把工具
能交付	第 5 章	敢给用户用	契约检查 + 两层验收 + bestRound
能托管	第 6 章	没人盯着也敢跑	四道护栏 + 五终态 + 诚实报告
能对话	第 7 章	敢让用户在旁边插话	SSE+HITL+ 意图识别 + 版本链
能进化	第 8 章	敢放手让它越用越好	上下文工程 + 四层记忆 + 偏好学习

方向变化： 3-6 章让机器更正确 → 7 章把人引进来 → 8 章让系统从人那里学到东西

每一次交互都留下痕迹，下次从更好的起点出发

"能进化"到底是什么意思

不是什么

不是在线微调
不是 RLHF
不改变模型权重

是什么

每次修改留痕迹
下次从更好的起点出发
不改代码不换模型就能进步

用户修改 → 偏好提取 → UserProfile → ContextAssembler 注入 → 输出更贴合 → 用户修改更少

这个闭环每转一圈，系统对这个人就好用一点

记忆只是手段，进化才是效果

双层 Loop 六器官 · 这一章的增量

器官	内层 Loop	外层 Loop	第八章新增
状态	Ledger+rounds	会话快照 + 版本链	+UserProfile(L3)
动作	调模型生成	意图→动作→权限	+ContextAssembler
观察	契约检查 + 验收	事件流 + 停止报告	+3 个新事件
判断	HARD 全过	问人：满意吗	+ 晋升三判据
反馈	problems→ 下轮	修改→下一版	+ 偏好→下次 prompt
停止	护栏 + 终态	接受 / 取消 / 过期	无新增

这一章的本质：把外层 Loop 断掉的反馈路径接上了

信息从 " 用户的不满意 " 一路流回到 " 下次模型的起点 "

核心判据速览 · 上下文工程

- ▶ 一条信息值不值得放进 prompt，看它会不会改变这一次的输出
- ▶ Context window 是有限预算：指令 > 约束 > 历史 > 画像
- ▶ 相关性过滤优先于压缩——先不放无关的
- ▶ 分区优先级管 "放不放"，物理顺序管 "放在哪"

记忆管理

- ▶ 记忆是素材库，上下文是消费者
- ▶ 频次 ≥ 3 + 方向一致 + 非上下文依赖，三条都过才算
- ▶ 记忆最危险的属性不是记不住，是忘不掉
- ▶ 记忆不是越多越好，是越准越好

核心判据速览 · 验证

成对断言

"记住了"和"忘掉了"
是一枚硬币的两面

最危险的错

"该忘的没忘"
比"该记的没记"更危险

第一条测试

不是"记住了吗"
是"记的是不是这个人"

个性化准确率

评测维度
不是可选指标

第二种错误无法被纠正，因为用户感知不到它的存在

这一章的交付物 · 账单

20

事件类型 (17→20)

15

评测维度 (7→15)

130

测试 (114→130)

150

评测用例 (50×3)

1

Prompt 四分区优先级表 + 四种压缩手段

选择性注入 / 摘要替代 / 滑动窗口 / 结构化

2

四层记忆 + 晋升遗忘机制

L1 工作 / L2 会话 / L3 用户 / L4 系统 + MemoryDecayPolicy

3

Prompt R + S + T

R 上下文组装 / S 偏好学习 / T 自动化评测

4

token 预算 context.max-tokens=6000

+ droppedSections 让 "丢了什么" 可观察

这些数字后面每一个都有判据

判据先行，实现后行

判据是可以争论的，
实现是不好回退的。

先定判据，你争论的成本是**一次会议**；
后定判据，你争论的成本是
一次重构。

从第 5 章贯穿到第 8 章的元原则——先立判据，再写代码

还缺什么 · 三个新缺口

效率与结构

四把刀串行 11 秒→7 秒白等
控制流藏在 if-else/while
不能审查、不能可视化

智能维度

一个大脑做所有决定
行程 / 预算 / 安全 / 推荐
认知负载超单 prompt 极限

生产维度

不知道今天比昨天好多少
成本 / 延迟 / 注入攻击
能 demo ≠ 能上线

下一章先解决第一个缺口 → 第九章：并行与图结构——Loop 是 Graph 的特例

三个缺口留给后面的章节逐一补上

课后思考 · 第九章的第一道题

留一个问题带走

你们的业务里，用户哪些行为是 " 临时的 "，哪些是 " 长期的 "？判据是什么？

如果只能用一种手段区分，你选频次还是显式确认？为什么？

正确答案几乎总是 " 按记错了的代价分级 "—— 代价大的用确认，代价小的用频次。

第七章教你怎么让人在机器跑着的时候安全地插手

这一章教你怎么让系统从人的每一次插手里学到东西

记忆只是手段，进化才是效果。

第九章见。